

Récurtivité

Warren McCulloch, avec Walter Pitts, est célèbre pour avoir posé les bases de la modélisation des neurones et de la logique formelle. Leur approche montrait que les réseaux de neurones simples pouvaient réaliser des fonctions logiques, et que certaines structures pouvaient simuler des processus récurtifs. La récurtivité, dans ce contexte, apparaît comme un mécanisme permettant à une structure simple de répéter des calculs ou de traiter des informations à plusieurs niveaux, inspirant plus tard les concepts d'algorithmes et de calculabilité.

Nicolas VACELET – Université Marie et Louis Pasteur

14 Octobre 2025

1 Théorème de Cormen-Leiserson-Rivest (Master Theorem)

Soit $a \geq 1, b > 1, f$ une fonction et $C(n)$ définie pour $n > n_0$ par :

$$C(n) = a C\left(\frac{n}{b}\right) + f(n)$$

alors :

1. Si $f(n) = O(n^{\log_b a - \varepsilon})$ pour une constante $\varepsilon > 0$, alors $C(n) = \Theta(n^{\log_b a})$
2. Si $f(n) = \Theta(n^{\log_b a})$, alors $C(n) = \Theta(n^{\log_b a} \log n)$

2 Fonction puissance

On considère deux méthodes pour calculer la puissance x^n :

$$\text{puissance1}(x, n) = \begin{cases} 1 & \text{si } n = 0, \\ x \cdot \text{puissance1}(x, n - 1) & \text{sinon.} \end{cases}$$

$$\text{puissance2}(x, n) = \begin{cases} 1 & \text{si } n = 0, \\ (\text{puissance2}(x, \lfloor n/2 \rfloor))^2 & \text{si } n > 0 \text{ et } n \text{ pair,} \\ x \cdot (\text{puissance2}(x, \lfloor n/2 \rfloor))^2 & \text{si } n > 0 \text{ et } n \text{ impair.} \end{cases}$$

1. Si $n = 20$, combien de multiplications effectue-t-on en appliquant `puissance1` ? Même question pour $n = 200$ et $n = 2000$.
2. Combien de multiplications effectue-t-on en appliquant `puissance2` pour $n = 20, n = 200$ et $n = 2000$?
3. Analyser vos résultats. Quelle est la complexité de chaque méthode en termes de nombre de multiplications ?
4. Laquelle des deux méthodes est la meilleure ?
5. On modifie maintenant le programme `puissance2` afin de minimiser le nombre de variables utilisées, obtenant la fonction suivante :

$$\text{puissance3}(x, n) = \begin{cases} 1 & \text{si } n = 0, \\ \text{puissance3}(x, \lfloor n/2 \rfloor) \cdot \text{puissance3}(x, \lfloor n/2 \rfloor) & \text{si } n > 0 \text{ et } n \text{ pair,} \\ x \cdot \text{puissance3}(x, \lfloor n/2 \rfloor) \cdot \text{puissance3}(x, \lfloor n/2 \rfloor) & \text{si } n > 0 \text{ et } n \text{ impair.} \end{cases}$$

Comment cette modification affecte-t-elle la complexité de l'algorithme ?

et que penser de

$$\text{puissance4}(x, n) = \begin{cases} 1 & n = 0, \\ (\text{puissance4}(x, \lfloor n/2 \rfloor))^2 & n > 0 \text{ pair}, \\ x \cdot (\text{puissance4}(x, \lfloor n/2 \rfloor))^2 & n > 0 \text{ impair}. \end{cases}$$

La fonction `puissance4` effectue un seul appel récursif pour $n > 0$, puis effectue une ou deux multiplications avec le résultat intermédiaire.

3 Les tours de Hanoï

Le problème des tours de Hanoï est le suivant : on dispose de 3 piquets A, B et C et de n disques de tailles différentes empilés sur le piquet A. L'objectif est de déplacer tous les disques du piquet A au piquet B en respectant les règles suivantes :

- On ne peut déplacer qu'un disque à la fois.
 - Un disque ne peut jamais être posé sur un disque plus petit.
1. Proposer une solution pour $n = 1, 2, 3, 4$ en indiquant la succession des déplacements.
 2. Proposer un algorithme récursif pour résoudre ce problème.
 3. Analyser le nombre de mouvements nécessaires en fonction de n .